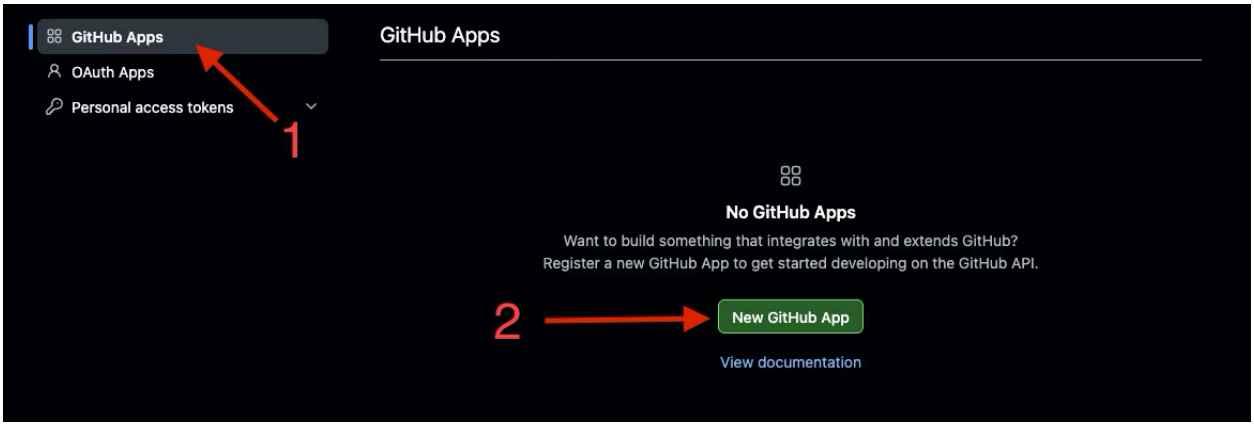
Instructions

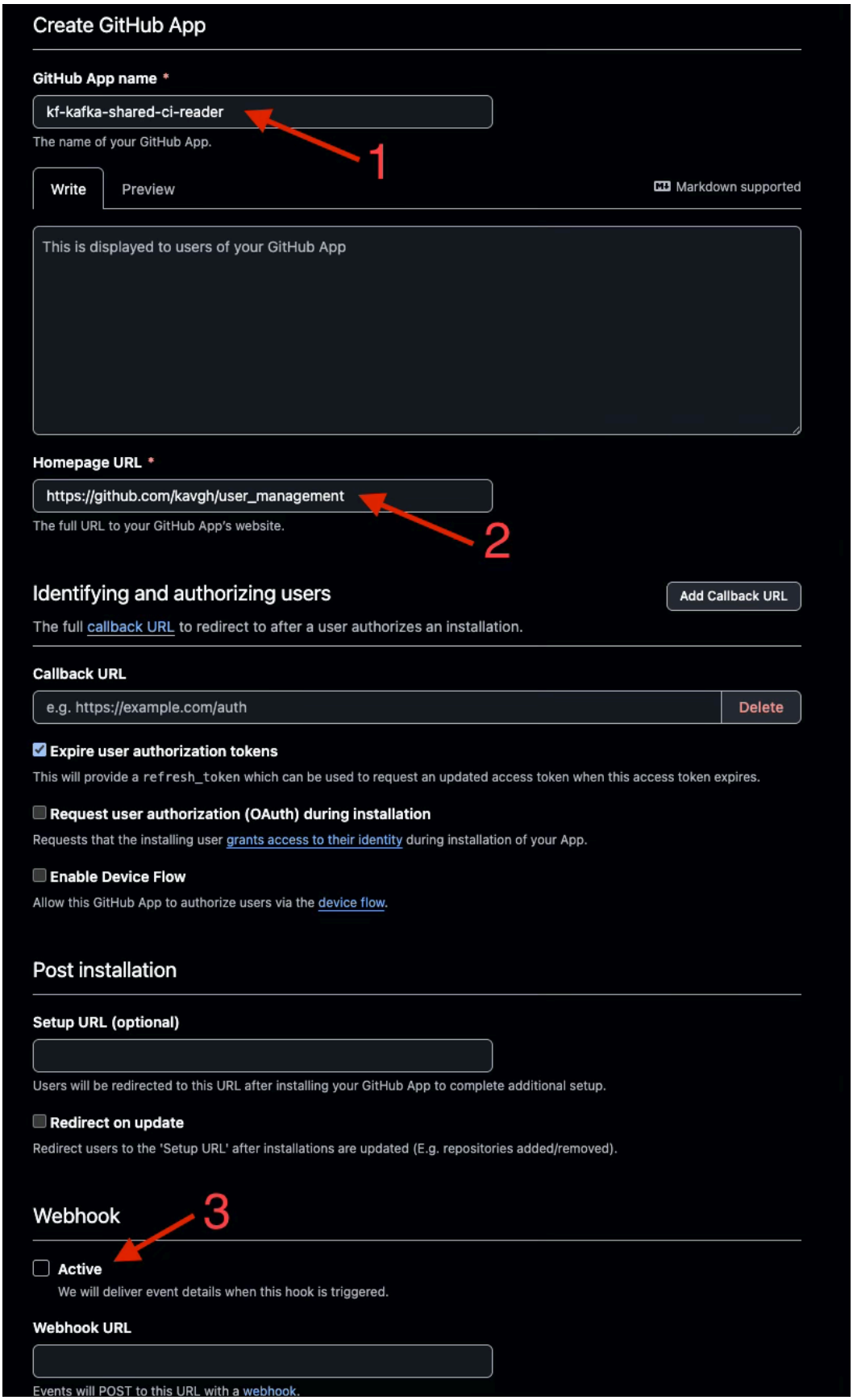
GitHub Application

1. Create the GitHub App

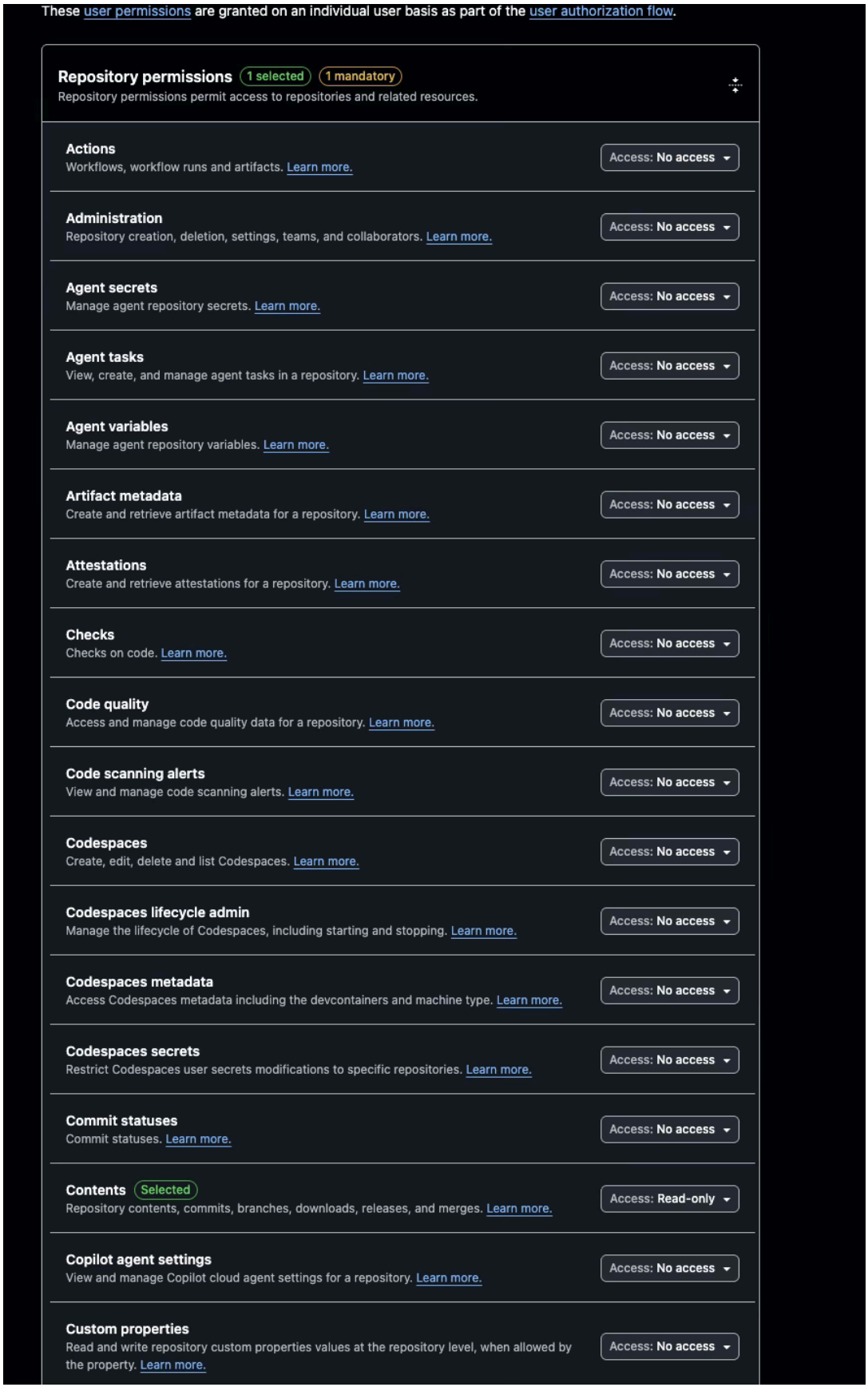
1. Go to the **organization** that owns `kf-kafka-shared` (this must match `github.repository_owner`, i.e. the same org as `user_management`) → **Settings** → **Developer settings** → **GitHub Apps** → **New GitHub App**.
    - Direct URL pattern: `https://github.com/organizations/<ORG>/settings/apps/new`



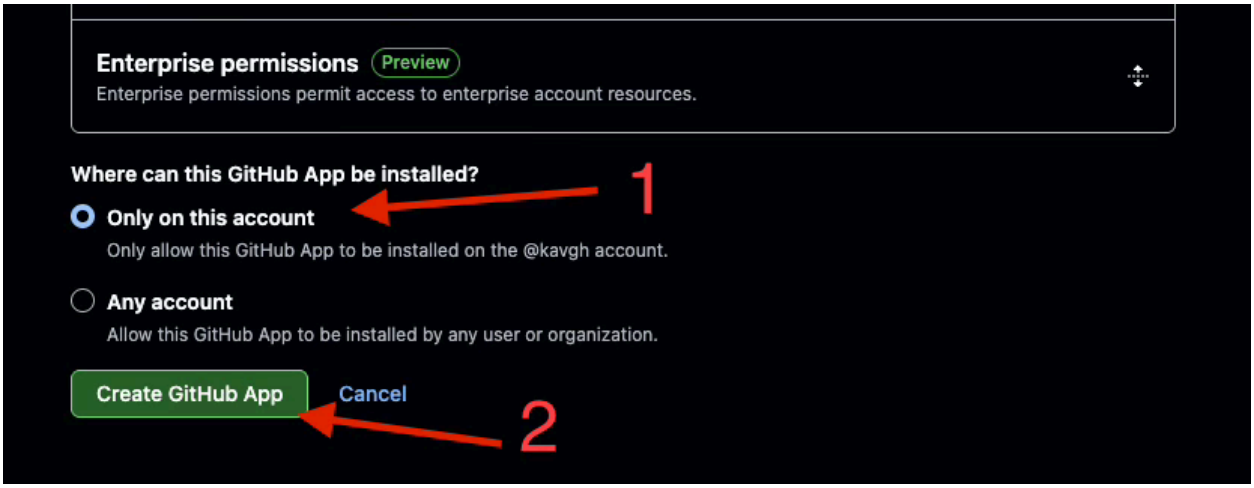
2. Fill in the required fields:
    - **GitHub App name**: something identifiable, e.g. `kf-kafka-shared-ci-reader`
    - **Homepage URL**: the org or repo URL (any valid URL works, it's not functionally used)
    - **Webhook**: uncheck **Active** — this app doesn't need webhooks


Create GitHub App
GitHub App name *
kf-kafka-shared-ci-reader
The name of your GitHub App.
1
Write
Preview
Markdown supported
This is displayed to users of your GitHub App
Homepage URL *
https://github.com/kavgh/user_management
The full URL to your GitHub App's website.
2
Identifying and authorizing users
Add Callback URL
The full callback URL to redirect to after a user authorizes an installation.
Callback URL
e.g. https://example.com/auth
Delete
Expire user authorization tokens
This will provide a refresh_token which can be used to request an updated access token when this access token expires.
Request user authorization (OAuth) during installation
Requests that the installing user grants access to their identity during installation of your App.
Enable Device Flow
Allow this GitHub App to authorize users via the device flow.
Post installation
Setup URL (optional)
Users will be redirected to this URL after installing your GitHub App to complete additional setup.
Redirect on update
Redirect users to the 'Setup URL' after installations are updated (E.g. repositories added/removed).
Webhook
3
Active
We will deliver event details when this hook is triggered.
Webhook URL
Events will POST to this URL with a webhook.

3. **Repository permissions**:
    - **Contents**: `Read-only` (this is the only permission required - it grants `git clone` / `git fetch` access)
    - Leave everything else as "No access"

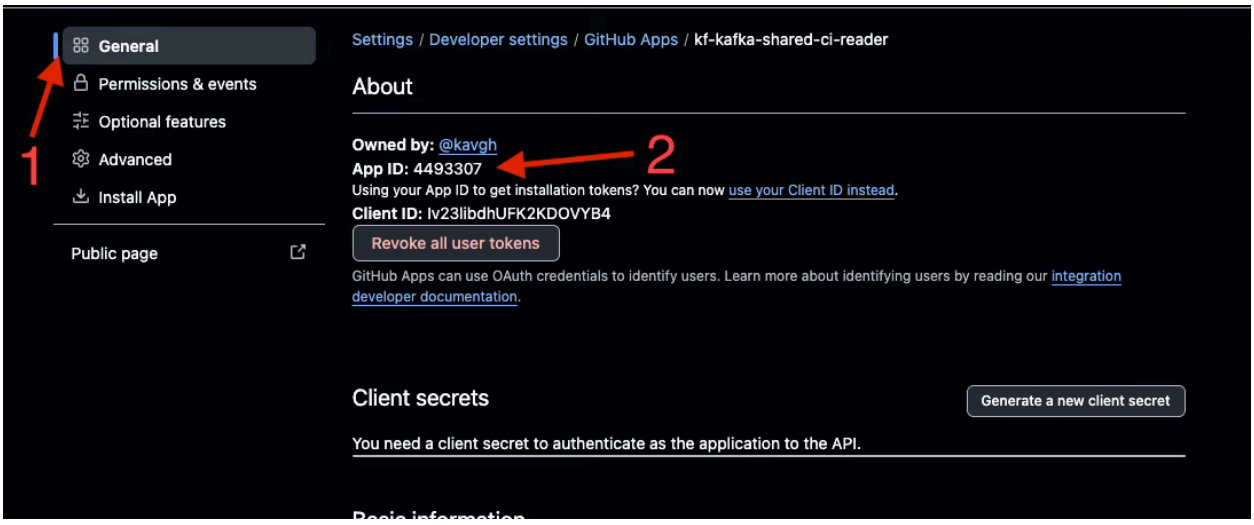

These user permissions are granted on an individual user basis as part of the user authorization flow.
Repository permissions 1 selected 1 mandatory
Repository permissions permit access to repositories and related resources.
Actions
Workflows, workflow runs and artifacts. Learn more.
Access: No access
Administration
Repository creation, deletion, settings, teams, and collaborators. Learn more.
Access: No access
Agent secrets
Manage agent repository secrets. Learn more.
Access: No access
Agent tasks
View, create, and manage agent tasks in a repository. Learn more.
Access: No access
Agent variables
Manage agent repository variables. Learn more.
Access: No access
Artifact metadata
Create and retrieve artifact metadata for a repository. Learn more.
Access: No access
Attestations
Create and retrieve attestations for a repository. Learn more.
Access: No access
Checks
Checks on code. Learn more.
Access: No access
Code quality
Access and manage code quality data for a repository. Learn more.
Access: No access
Code scanning alerts
View and manage code scanning alerts. Learn more.
Access: No access
Codespaces
Create, edit, delete and list Codespaces. Learn more.
Access: No access
Codespaces lifecycle admin
Manage the lifecycle of Codespaces, including starting and stopping. Learn more.
Access: No access
Codespaces metadata
Access Codespaces metadata including the devcontainers and machine type. Learn more.
Access: No access
Codespaces secrets
Restrict Codespaces user secrets modifications to specific repositories. Learn more.
Access: No access
Commit statuses
Commit statuses. Learn more.
Access: No access
Contents Selected
Repository contents, commits, branches, downloads, releases, and merges. Learn more.
Access: Read-only
Copilot agent settings
View and manage Copilot cloud agent settings for a repository. Learn more.
Access: No access
Custom properties
Read and write repository custom properties values at the repository level, when allowed by the property. Learn more.
Access: No access

4. **Where can this GitHub App be installed?**: choose **Only on this account** (keeps it private to your org)
5. Click **Create GitHub App**.



2. Get the App ID

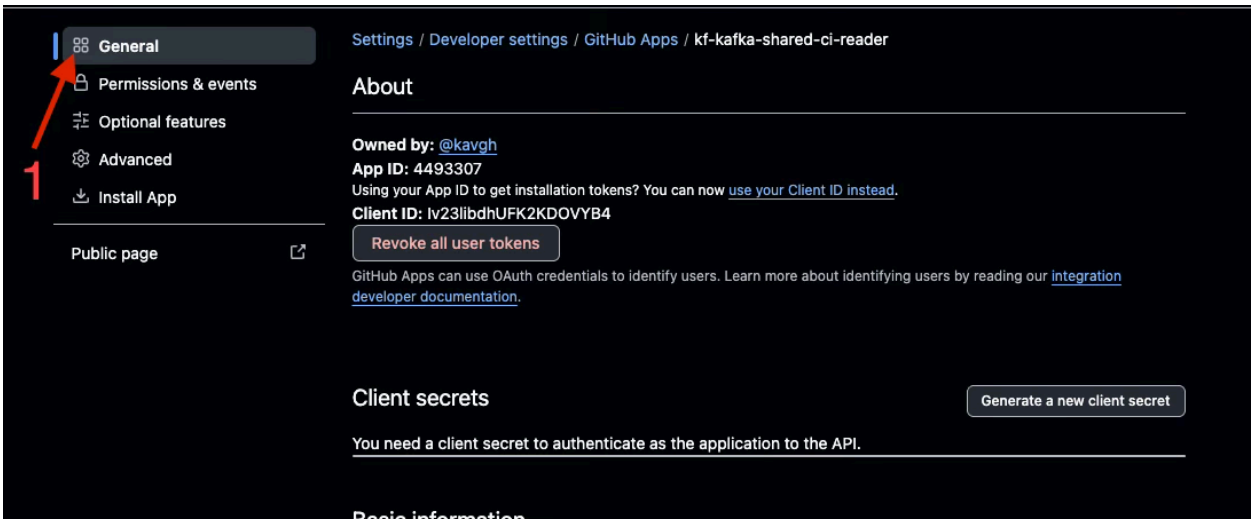
After creation you land on the app's settings page. Copy the **App ID** shown at the top - you'll need it for `GH_CLIENT_ID`.

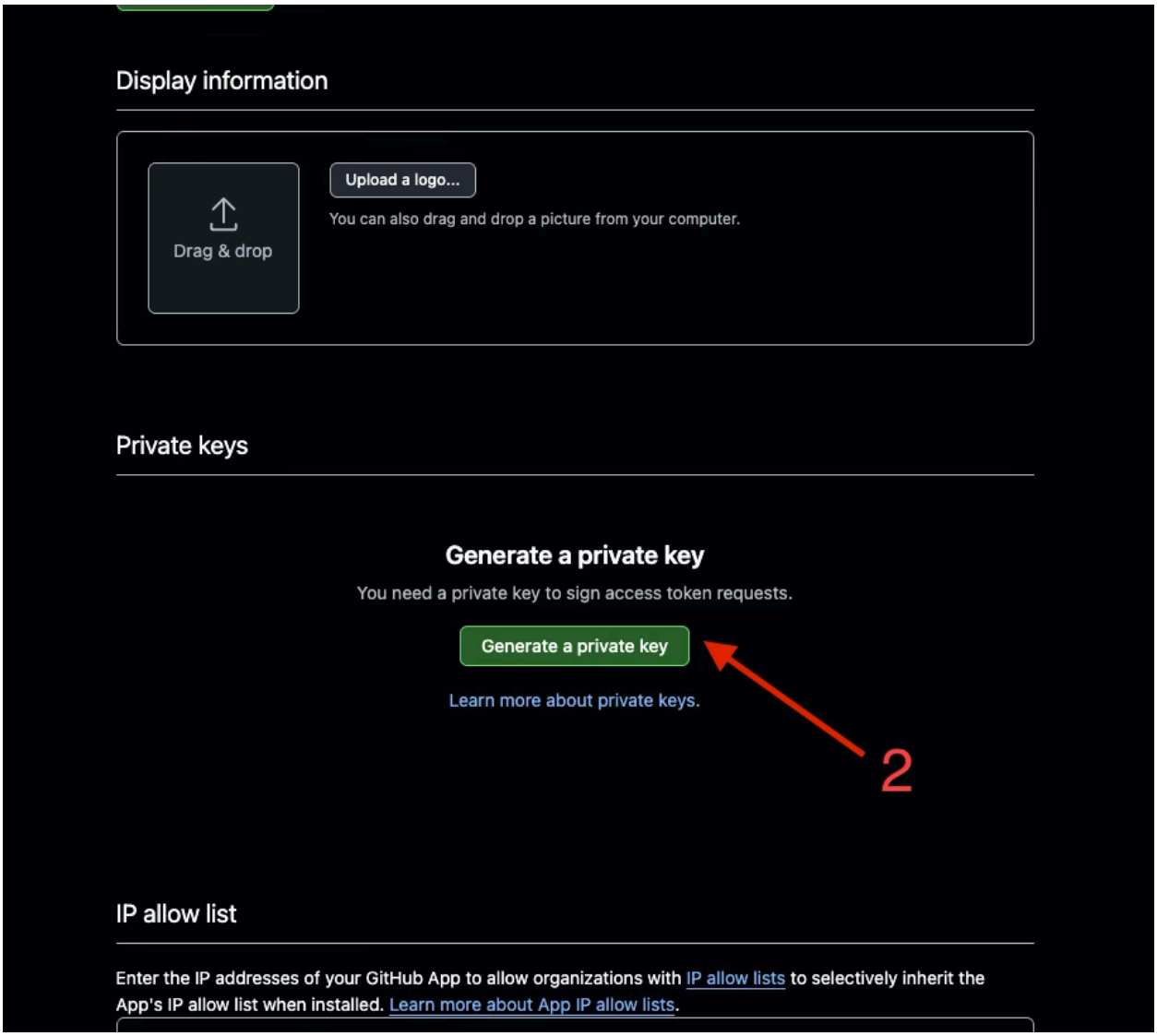


3. Generate a private key

On the same settings page, scroll to **Private keys** → **Generate a private key**. This downloads a `.pem` file - save it securely, it can't be re-downloaded (only

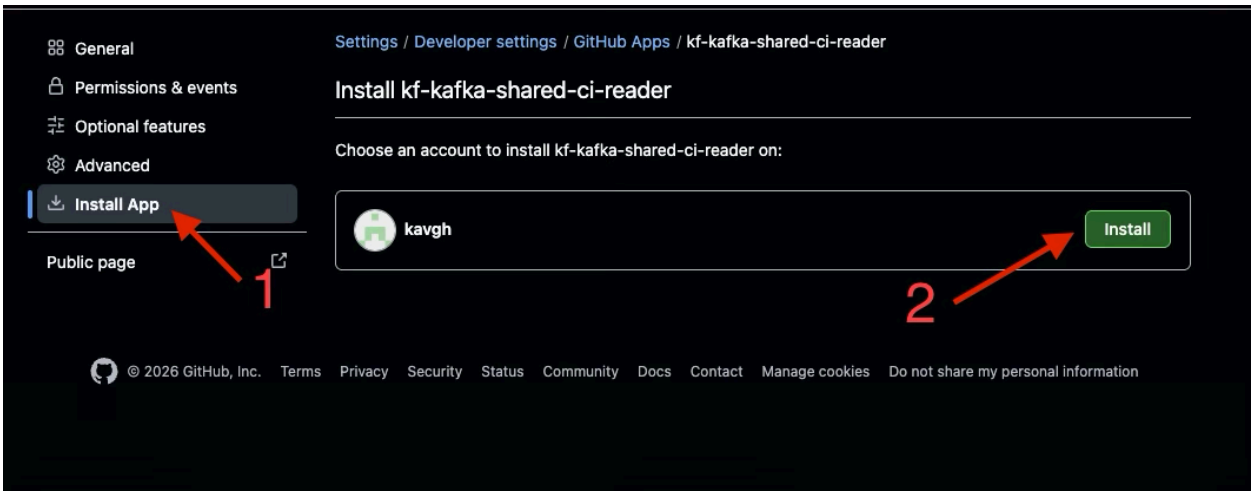
regenerated).

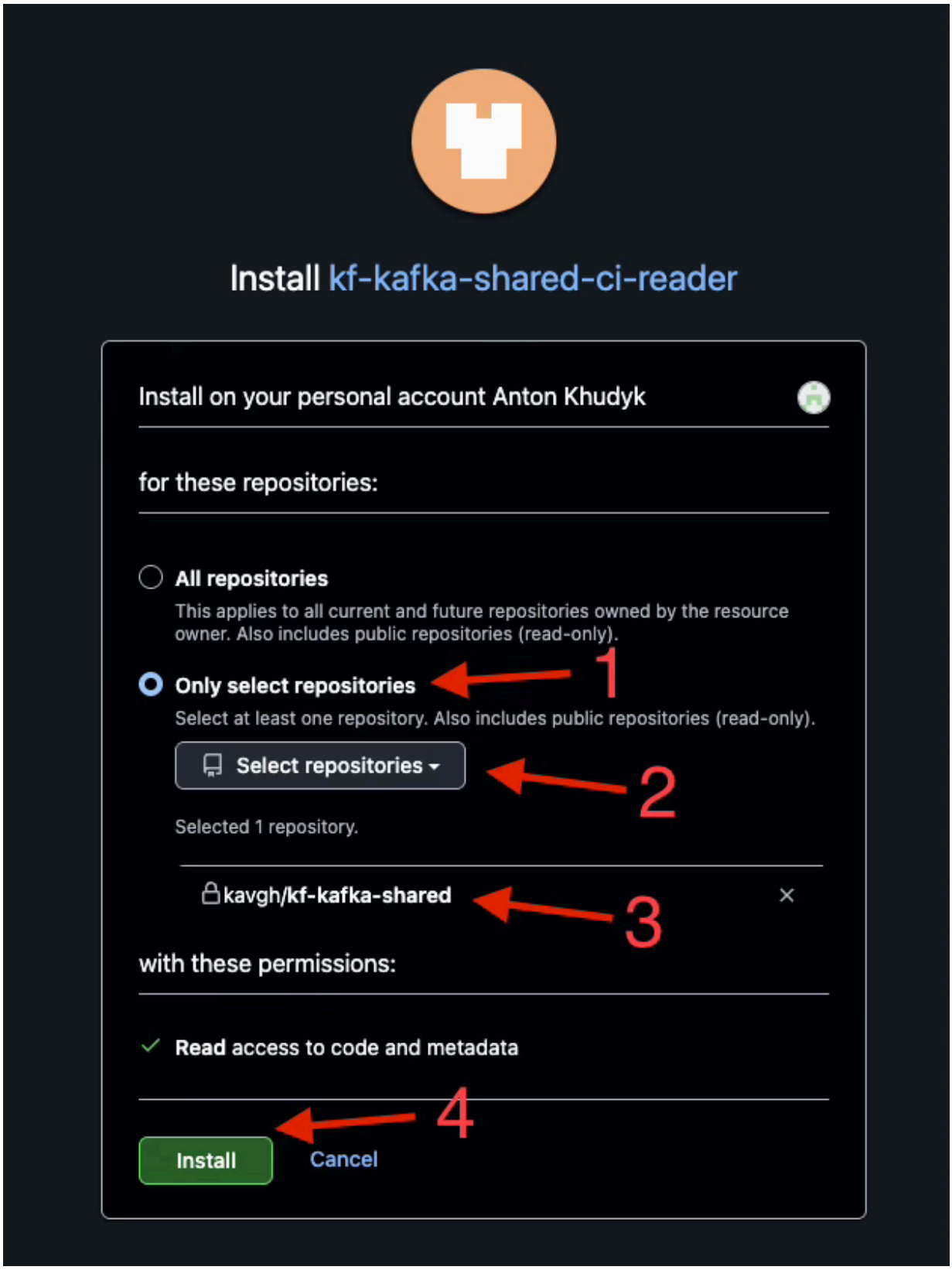




4. Install the app

1. In the left sidebar of the app's settings, click **Install App**.
2. Install it on the org, choosing **Only select repositories**, and select `kf-kafka-shared` (matches the `repositories: kf-kafka-shared` input in the workflow).
    - You do **not** need to also select `user_management` - the token is only used to authenticate the clone of `kf-kafka-shared`.


General
Permissions & events
Optional features
Advanced
Install App
Public page
Settings / Developer settings / GitHub Apps / kf-kafka-shared-ci-reader
Install kf-kafka-shared-ci-reader
Choose an account to install kf-kafka-shared-ci-reader on:
kavgh
Install
1
2
© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information


Install kf-kafka-shared-ci-reader
Install on your personal account Anton Khudyk
for these repositories:
All repositories
This applies to all current and future repositories owned by the resource owner. Also includes public repositories (read-only).
Only select repositories
1
Select at least one repository. Also includes public repositories (read-only).
Select repositories
2
Selected 1 repository.
kavgh/kf-kafka-shared
3
with these permissions:
Read access to code and metadata
4
Install
Cancel

5. Add the App ID and private key to user_management

In the `user_management` repo → **Settings** → **Secrets and variables** → **Actions**:

- **Variables tab** → New repository variable:
  - Name: `GH_CLIENT_ID`
  - Value: the App ID from step 2
- **Secrets tab** → New repository secret:
  - Name: `GH_APP_PRIVATE_KEY`
  - Value: paste the **entire contents** of the downloaded .pem file (including the `------BEGIN/END RSA PRIVATE KEY-----` lines)

These map directly to `vars.GH_CLIENT_ID` and `secrets.GH_APP_PRIVATE_KEY`

6. Verify

- Trigger the workflow (push to `main` or `workflow_dispatch`).
- In the `build` job, the **Generate GitHub App token** step should succeed and produce a masked token.
- If it fails with a 404/permission error, double-check the app is installed with access to `kf-kafka-shared` specifically, and that `GH_CLIENT_ID` / `GH_APP_PRIVATE_KEY` are set on the `user_management` repo (not just the org, unless you intend org-level secrets).